

Lab Practice Exercises 4

Lectures 6 and 7 · Practice only — nothing here is collected or graded

Fall Semester 2026 · Dr. Narasimha Rao Vajjhala · nrao@aubg.edu · Office MB 253

BEFORE YOU START

Environment	JDK 25 (the current Long-Term Support release) and IntelliJ IDEA — the same setup as Labs 1 to 3. Check it with <code>javac -version</code> before Question 1.
What this covers	Lecture 6 (arrays: indexing, the processing idioms, copying and <code>java.util.Arrays</code> , arrays and methods, varargs, command-line arguments, two dimensions) and Lecture 7 (Strings: the core methods, characters, equality, immutability, <code>StringBuilder</code> , and <code>split</code> , <code>matches</code> and <code>replaceAll</code>).
Feeding the samples in	As before, redirect a file: <code>java Program.java < input.txt</code> . Question 8 takes its numbers from the command line instead — <code>java Stats.java 4 8 15 16 23 42</code> — and in IntelliJ they go in the run configuration's Program arguments field.
Style	Import <code>java.util.Arrays</code> and print arrays with <code>Arrays.toString</code> — <code>println(list)</code> prints a type code and a hash code, not the elements (Question 21). Give every method a <code>/** */</code> comment and brace every <code>if</code> and loop body, as in Lab 3.
Nothing to hand in	These exercises are for practice. There is nothing to submit, no deadline and no mark — keep your <code>.java</code> files somewhere you can find them again, and bring anything that will not work to the lab or to office hours.

Questions 1 to 20 are the core set: 1 to 13 are arrays, from Tuesday's lecture, and 14 to 20 are Strings, from Thursday's. All five of the arrays lecture's practice programs are here — `AnalyzeNumbers` in 1, `reverse` in 2, the matrices in 9, `GradeExam` in 10 and `countLetters` inside the anagram test of 15. Questions 21 to 30 go deeper into what the core set uses: arrays as references, the errors everyone meets in the first week of arrays, ragged arrays, varargs and `main`'s own array, and the places where Strings surprise people — equality, immutability, `compareTo` and index ranges, `split`, and the cost of `+=` against `StringBuilder`.

None of this is collected, so work at whatever pace suits you and skip anything you already find easy. Two suggestions all the same. Trace an array on paper before you run anything that changes one — the lecture's table, one row per pass and one column per element, settles most arguments about what a loop does. And predict before you run in Part B.

Arrays and Strings are not on Quiz 1 this Thursday, which covers Weeks 1 to 3; they are Quiz 2 material (22.10). Two questions go a little beyond the lectures: 6 writes selection sort and binary search (Liang, Chapter 7), and 7 measures the shuffle on Lecture 6's slide 14 against a provably fair one.

Part A · Core Exercises

Question 1 · Analyze Numbers, Extended

Read `n`, then `n` numbers into a `double[]`. Write one method per job — `mean`, `standardDeviation`, `min`, `max` and `countAbove(double[] values, double threshold)` — and print the mean, the sample standard deviation, the smallest and largest values, and how many values lie above the mean. This is the lecture's `AnalyzeNumbers` with the rest of its practice slide added.

The sample standard deviation is the square root of $\sum(x - \text{mean})^2 / (n - 1)$. It needs the mean before it can start — and so does `countAbove` — which is exactly why the values have to be kept in an array rather than read and forgotten.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of values: 8
Enter 8 values: 72.5 88 91 64 79.5 85 70 93
Mean: 80.38
Standard deviation: 10.63
Smallest: 64.0
Largest: 93.0
Above the mean: 4 of 8
```

Seed `min` and `max` from `values[0]`, never from 0. With -3, -8 and -1, a maximum seeded with 0 reports 0 — a value that is not in the array. That is the warning on the lecture's slide 12, now with a test that proves it.

Question 2 · Reversing an Array, Two Ways

Write `public static int[] reverse(int[] list)`, which returns a new array and leaves its argument alone — the lecture's version — and `public static void reverseInPlace(int[] list)`, which swaps elements from both ends towards the middle and returns nothing.

Then write the palindrome test twice: `isPalindrome(int[] list)` with `reverse` and `Arrays.equals`, and `isPalindromeInPlace(int[] list)` with two indices and no second array. The program reads an array and exercises all four methods.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of values: 7
Enter 7 values: 3 1 4 1 5 9 2
reverse:          [2, 9, 5, 1, 4, 1, 3]
original after it: [3, 1, 4, 1, 5, 9, 2]
palindrome?       false / false
after reverseInPlace: [2, 9, 5, 1, 4, 1, 3]

Enter the number of values: 5
Enter 5 values: 1 2 3 2 1
reverse:          [1, 2, 3, 2, 1]
original after it: [1, 2, 3, 2, 1]
palindrome?       true / true
after reverseInPlace: [1, 2, 3, 2, 1]
```

`reverseInPlace` needs only `list.length / 2` swaps. Run the loop to the full length and every pair is swapped twice: the array ends up exactly as it started, and nothing tells you so.

Question 3 · Distinct Values in Order of First Appearance

Write `public static int[] distinct(int[] list)`, which returns each value of `list` once, in the order it first appears, in a new array exactly as long as the number of distinct values.

Fill a working array as long as `list`, keep a count of how much of it you have used, and trim it with `Arrays.copyOf(work, count)` at the end. A helper `contains(int[] list, int size, int key)` that searches only the used part keeps `distinct` short.

EXPECTED OUTPUT (EXAMPLE)

```
Enter ten numbers: 1 2 3 2 1 6 3 4 5 2
Distinct values: [1, 2, 3, 6, 4, 5]
6 of 10 values are distinct
```

A partially filled array always travels with a count: the array's length says how much room there is, the count says how much of it holds data. Search all of `work` instead of its first `count` elements, and a 0 typed by the user is taken for a duplicate of an empty slot.

Question 4 · Rotating an Array by k Places

Write `public static void rotateRight(int[] list, int k)`, which moves every element `k` places to the right in place, the last `k` wrapping round to the front. `k` may be larger than the length, and a negative `k` rotates to the left.

Build it on the lecture's shift idiom (slide 14), shifting right this time: a method `shiftRightByOne` that saves the last element, moves the others along and puts the saved one at index 0. Remember the slide's warning — shifting right, walk backwards.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of values: 7
Enter 7 values: 10 20 30 40 50 60 70
Enter k: 3
Right by 3: [50, 60, 70, 10, 20, 30, 40]
Right by 10: [50, 60, 70, 10, 20, 30, 40]
Right by -3: [40, 50, 60, 70, 10, 20, 30]
```

Reduce `k` before you loop: rotating by `k + n` is the same as rotating by `k`. For a negative `k`, Java's `%` gives a negative answer — `-3 % 7` is `-3` — while `Math.floorMod(-3, 7)` is `4`, which is exactly a rotation of `3` to the left.

Question 5 · Exam Score Distribution

Lab 2's Question 18 counted grades with five named counters. With an array, the number of categories stops mattering. Write `public static int[] distribution(int[] scores)`, which counts whole-number scores into ten bins — `0–9`, `10–19` and so on up to `90–100` — with `counts[Math.min(score / 10, 9)]++`, and `public static void printHistogram(int[] counts)`.

The program reads the number of scores, then the scores, and prints the histogram.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of scores: 15
Enter 15 scores: 95 82 67 70 58 88 90 100 45 73 77 81 64 99 38
0-9 | 0
10-19 | 0
20-29 | 0
30-39 | * 1
40-49 | * 1
50-59 | * 1
60-69 | ** 2
70-79 | *** 3
80-89 | *** 3
90-100 | **** 4
```

Leave out the `Math.min` and the program dies on the one score that needs it: `100 / 10` is `10`, and the message is `Index 10 out of bounds for length 10`. Test the boundaries — `0`, `9`, `10`, `99` and `100` — not just the typical values.

Question 6 · Selection Sort and Binary Search

`Arrays.sort` and `Arrays.binarySearch` do this for you; writing them once is how you learn what they promise. Write `public static void selectionSort(int[] list)` — on each pass, find the smallest remaining element and swap it into place — and `public static int binarySearch(int[] list, int key)`, which returns the index of `key` in a sorted array, or `-(insertion point) - 1` when `key` is absent, exactly as `Arrays.binarySearch` does.

The program sorts one copy with your method and another with `Arrays.sort` and compares them, searches for two keys, and finally checks your search against the library's for every key from `0` to `100`.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of values: 10
Enter 10 values: 64 25 12 22 11 90 3 47 36 58
Sorted: [3, 11, 12, 22, 25, 36, 47, 58, 64, 90]
Same as Arrays.sort: true
Enter a key to search for: 47
47 is at index 6
Enter a key to search for: 50
50 is not there: binarySearch returned -8, so it belongs at index 7
Keys 0 to 100 against Arrays.binarySearch: 0 disagreements
```

The `- 1` in `-(insertion point) - 1` is there for one case. A key smaller than everything belongs at index 0, and `-0` is 0 — which would read as "found at index 0". With the `- 1`, every "not found" answer is negative.

Question 7 · Is the Shuffle Fair?

The shuffle on Lecture 6's slide 14 swaps each element with one chosen from the whole array. Shuffle {1, 2, 3} 600,000 times with it and count how often each of the six possible orders comes up, using an `int[6]` indexed by `public static int orderIndex(int[] a): [1, 2, 3] is 0, [1, 3, 2] is 1, and so on to [3, 2, 1], which is 5. A fair shuffle gives each order about 100,000.`

Then change one thing — choose `j` from 0 to `i` instead of from 0 to `n - 1`, which makes it the Fisher–Yates shuffle — and count again.

EXPECTED OUTPUT (ONE RUN)

```
The slide's shuffle (j from 0 to n - 1):
[1, 2, 3] 89179 14.86%
[1, 3, 2] 111554 18.59%
[2, 1, 3] 111264 18.54%
[2, 3, 1] 110473 18.41%
[3, 1, 2] 88847 14.81%
[3, 2, 1] 88683 14.78%
Fisher-Yates (j from 0 to i):
[1, 2, 3] 100018 16.67%
[1, 3, 2] 99846 16.64%
[2, 1, 3] 99512 16.59%
[2, 3, 1] 100716 16.79%
[3, 1, 2] 100224 16.70%
[3, 2, 1] 99684 16.61%
```

The slide's loop makes three choices of three positions: 27 equally likely runs, to be shared among 6 orders. 27 is not a multiple of 6, so some orders must get 5 runs and others 4, however good `Math.random` is — 18.5% against 14.8%. The Fisher–Yates loop makes $1 \times 2 \times 3 = 6$ runs, one per order; three nested loops over the choices will show you which orders get 5.

Question 8 · Statistics from the Command Line

Write a program `Stats.java` whose numbers arrive as command-line arguments rather than through a `Scanner`. Convert each `args[i]` with `Integer.parseInt`, and compute with three varargs methods: `public static int sum(int... values)`, `public static double average(int... values)` and `public static int max(int first, int... rest)`.

With no arguments at all, print a usage line and stop: `args` is then an empty array, never `null`.

EXPECTED OUTPUT (EXAMPLE)

```
java Stats.java 4 8 15 16 23 42
Count: 6
Sum: 108
Average: 18.00
Max: 42
```

```
java Stats.java
Usage: java Stats.java n1 n2 ...
```

`max(int first, int... rest)` cannot be called with nothing — `max()` does not compile — which is right for a maximum. `sum()` is 0 and `average()` is NaN, which may not be. The same header means `max(numbers)` with an `int[]` will not compile either: pass `numbers[0]`, and `Arrays.copyOfRange(numbers, 1, numbers.length)` for the rest.

Question 9 · Matrix Operations

Write `public static int[][] add(int[][] a, int[][] b)`, `public static int[][] multiply(int[][] a, int[][] b)` and `public static int[][] transpose(int[][] m)`, each returning a new matrix, and `public static void print(int[][] m)`, which prints a matrix in aligned columns. Read two 3×3 matrices and print their sum and their product.

Then use `Arrays.deepEquals` to check two facts: the transpose of AB equals the transpose of B times the transpose of A , and AB is not in general equal to BA .

EXPECTED OUTPUT (EXAMPLE)

```
Enter matrix A (3 x 3):
1 2 3
4 5 6
7 8 9
Enter matrix B (3 x 3):
0 2 4
1 3 5
2 4 6
A + B:
1 4 7
5 8 11
9 12 15
A x B:
8 20 32
17 47 77
26 74 122
(A x B) transposed equals B^T x A^T: true
A x B equals B x A: false
```

Element $[i][j]$ of the product is the sum over k of $a[i][k] * b[k][j]$ — a third loop, inside the two you already have. Take every bound from the arrays themselves (`a.length`, `b[0].length`, `b.length`), and the same method multiplies a 2×3 by a 3×4 unchanged.

Question 10 · Grading a Multiple-Choice Test

Extend the lecture's GradeExam (slide 30) to eight students. The first three rows of answers are the slide's. For each student print the number of correct answers; then print the class average, the number of students who answered each question correctly, and the hardest question.

THE ANSWERS AND THE KEY

```
// each row is one student, each column one question (rows 0 to 2 are the lecture's)
char[][] answers = {
    {'A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
    {'D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'},
    {'E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'},
    {'D', 'B', 'D', 'C', 'B', 'D', 'E', 'E', 'A', 'C'},
    {'D', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
    {'B', 'B', 'A', 'C', 'C', 'A', 'E', 'A', 'A', 'D'},
    {'D', 'C', 'D', 'C', 'A', 'D', 'E', 'E', 'B', 'D'},
    {'C', 'B', 'D', 'A', 'C', 'D', 'E', 'E', 'A', 'D'}
};
char[] keys = {'D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D'};
```

Write the work as methods: `int[] scores(char[][] answers, char[] keys)`, `int[] correctPerQuestion(char[][] answers, char[] keys)`, `double average(int[] values)` and `int indexOfMin(int[] values)`.

EXPECTED OUTPUT (EXAMPLE)

```
Student 0: 7 correct
Student 1: 6 correct
Student 2: 5 correct
Student 3: 7 correct
Student 4: 9 correct
Student 5: 5 correct
Student 6: 6 correct
Student 7: 7 correct
Class average: 6.50 out of 10
Correct answers per question: [4, 6, 5, 5, 6, 5, 0, 7, 7, 7]
Hardest question: 7 (index 6), 0 of 8 correct
```

Nobody answered question 7 correctly: every student wrote E, and the key says A. When a whole class agrees on one wrong answer, check the key before you check the students — and a program that reports per-question counts is how you notice.

Question 11 · Quarterly Sales by Region

A company records its sales, in thousands of euros, for four regions over four quarters: a `double[4][4]` with one row per region, and the region names in a `String[]`. Write `double[] rowTotals(double[][] m)`, `double[] columnTotals(double[][] m)`, `int indexOfMax(double[] values)` and `int[] locateLargest(double[][] m)`, which returns the row and column of the largest single figure as a two-element array.

Print the table with a total for each region and each quarter, then the largest single figure and where it is, the best region and the best quarter.

EXPECTED OUTPUT (EXAMPLE)

```
Sales for Sofia (Q1-Q4): 120.5 98.0 143.2 160.8
Sales for Plovdiv (Q1-Q4): 80.2 85.6 91.0 99.4
Sales for Varna (Q1-Q4): 45.0 70.3 112.8 60.1
Sales for Burgas (Q1-Q4): 38.7 66.9 104.5 52.3
Region      Q1      Q2      Q3      Q4      Total
Sofia       120.5   98.0   143.2   160.8   522.5
Plovdiv     80.2    85.6   91.0    99.4    356.2
Varna       45.0    70.3   112.8   60.1    288.2
Burgas      38.7    66.9   104.5   52.3    262.4
Total       284.4   320.8   451.5   372.6   1429.3
Largest single figure: 160.8 (Sofia, Q4)
Best region: Sofia
Best quarter: Q3
```

`locateLargest` returns two numbers in one array — the only way a method can hand back two values until classes arrive in Week 5. Lab 3's swap could not be written as a method; this can.

Question 12 · Pascal's Triangle as a Ragged Array

Build Pascal's triangle with n rows as an `int[][]` in which row i has exactly $i + 1$ elements: create the outer array with `new int[n][]`, then each row with its own new `int[i + 1]`. Every row starts and ends with 1, and every other element is the sum of the two above it.

Print the rows, their sums — which should be the powers of 2 — and the lengths of the outer array and of the last row.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of rows: 7
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
Row sums: 1 2 4 8 16 32 64
triangle.length is 7, triangle[6].length is 7
```

Ask for 40 rows. The row sums break first: row 31's should be 2^{31} , one more than `Integer.MAX_VALUE`, and prints as -2147483648, and every later sum prints as 0. From row 34, counting from 0, the entries overflow too — some turn negative, others are wrong and look fine. With `long` entries the triangle holds up to row 66.

Question 13 · Checking a Sudoku Solution

Slide 31 showed only the top of the Sudoku checker. Write the four helpers it calls: `boolean hasAllNine(int[] values)` — true if the nine values are 1 to 9 in some order, which a `boolean[10]` of "seen" flags settles in one pass; `int[] row(int[][] grid, int r)`; `int[] column(int[][] grid, int c)`; and `int[] box(int[][] grid, int r, int c)`, the nine values of the 3×3 box whose top-left corner is (r, c) .

Copy `isValid` from the slide, read a grid of nine lines, and report whether it is a valid solution.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the grid, one row per line:
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
Valid solution
```

Test it with a grid that should fail in only one way. Swap two whole rows from different bands — `grid[2]` and `grid[3]`, say — and every row still holds 1 to 9, as does every column, which has only changed order; only the box check catches it. A test that only one of your three checks can fail is the best test of that check.

Question 14 · Parsing Student Records

Each line of a class list holds a surname and first name, a course and a score, separated by semicolons, with untidy spaces anywhere: Ivanova, Elena ; COS 2030 ; 91. Read lines until one says END and print each record tidily — Elena Ivanova (COS 2030): 91, A- — followed by the number of records and their average.

Use `split` twice (on ";" and then on ","), `strip` on every piece, `Integer.parseInt` for the score, and `LetterGrade` from Lab 3's Question 2. Collect the report in a `StringBuilder` and print it after END.

EXPECTED OUTPUT (EXAMPLE)

```
Enter records as surname, first name ; course ; score
and END to finish:
Ivanova, Elena ; COS 2030 ; 91
Petrov,Georgi;COS 2030;78
Okafor , Chidi ;INF 1030; 96
END
Elena Ivanova (COS 2030): 91, A-
Georgi Petrov (COS 2030): 78, C+
Chidi Okafor (INF 1030): 96, A
3 records, average 88.33
```

Leave out one `strip()` and `Integer.parseInt(" 91")` stops the program with `NumberFormatException`: For input string: " 91" — the space is part of the string. The quotation marks in that message are there to show you exactly such spaces.

Question 15 · Letter Counts and Anagrams

Write `public static int[] letterCounts(String s)`, which returns an `int[26]` holding how often each letter a to z occurs in `s`, ignoring case and every other character — the lecture's `countLetters`, with `ch - 'a'` as the index. Use it for `public static boolean isAnagram(String a, String b)`: two phrases are anagrams when their counts are equal, which `Arrays.equals` decides.

Add `public static String describe(int[] counts)`, which lists the non-zero counts as `d1 i1 m1 o2`, and print it for both phrases.

EXPECTED OUTPUT (EXAMPLE)

```
First phrase: Dormitory
Second phrase: Dirty room!
Letters in the first: d1 i1 m1 o2 r2 t1 y1
Letters in the second: d1 i1 m1 o2 r2 t1 y1
Anagrams: true
```

Test for a letter with `ch >= 'a' && ch <= 'z'` after lower-casing, not with `Character.isLetter`. The Bulgarian letter `ж` is a letter too — and `'ж' - 'a'` is 981, far outside an `int[26]`.

Question 16 · Sentence Palindromes with StringBuilder

Lab 2's Question 12 tested a palindrome with two indices. Do it again the Lecture 7 way: `public static String lettersAndDigits(String s)` builds, in a `StringBuilder`, the letters and digits of `s` in lower case, and `public static boolean isPalindrome(String s)` compares that string with its reverse.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a line of text: Was it a car or a cat I saw?
Filtered: wasitacaroracatisaw
Palindrome
```

`sb.reverse()` reverses `sb` itself and returns it. So `sb.reverse().toString().equals(sb.toString())` compares the reversed text with itself and is true for every input. Reverse a copy: `new StringBuilder(filtered).reverse().toString()`.

Question 17 · Word Statistics

Read a line and report the number of words, the longest word, the average word length, and the words in reverse order — "Java is fun" becomes "fun is Java", as on the lecture's practice slide. Split with `strip().split("\\s+")`, so that any run of spaces counts as one separator, and write `longest`, `averageLength` and `reverseOrder` as methods that take the `String[]`.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a sentence:   Java   is fun and methods make it tidy
Words: 8
Longest: methods (7 letters)
Average length: 3.63
Reversed: tidy it make methods and fun is Java
```

Leave out `strip()` and a line that starts with spaces splits into an empty first word: `split` removes empty strings from the end of its result but never from the start, so `" Java is fun ".split("\\s+")` has 4 elements, the first of them `""`. Guard against a blank line as well — `"".split("\\s+")` is one empty word, not none.

Question 18 · Run-Length Encoding

Run-length encoding replaces each run of a repeated character with the character and the length of the run: "aaabcc" becomes "a3b1c2", the lecture's practice example. Write `public static String encode(String s)` and `public static String decode(String s)` with `StringBuilder`; decode must read counts of more than one digit, so that "x12" is twelve x's.

Print the encoded form, both lengths, and whether decoding gives the original back.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a line: WWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWB
Encoded: W12B1W12B3W24B1W14
Length 67 -> 18
Round trip: true
```

Encoding "abc" gives "a1b1c1", twice as long: the scheme only pays on runs. And a digit in the input breaks it — "a1" encodes to "a111", which decodes to 111 a's. Say in the `/** */` comment which inputs the methods accept; that sentence is part of the method.

Question 19 · Caesar Cipher with Wrap-Around

Write `public static char shift(char ch, int k)`, which moves a letter `k` places along the alphabet, wrapping from `z` back to `a` and keeping its case; any other character comes back unchanged. Then write `public static String encrypt(String text, int k)`, and `decrypt`, which is nothing but `encrypt` with `-k`.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the text: Meet me at MB 253, 17:45!
Enter the shift: 3
Encrypted: Phhw ph dw PE 253, 17:45!
Decrypted: Meet me at MB 253, 17:45!
```

With a negative shift, `(ch - 'a' + k) % 26` goes negative: decrypting 'a' by 3 gives -3, and `(char) ('a' - 3)` is '^'. `Math.floorMod` keeps the result between 0 and 25. The cast is needed because `char + int` is an `int`.

Question 20 · Password Policy Checker

A password must have at least 10 characters, an upper-case letter, a lower-case letter, a digit, and a symbol — a character that is not a letter, a digit or white space — and it must not contain spaces. Check the length directly, and write one boolean method for each of the other five rules using `Character` methods in a loop — `hasUpper`, `hasLower`, `hasDigit`, `hasSymbol` and `hasSpace`. Report every rule the password breaks, not just the first.

Then write the same five checks as regular expressions with `matches` — `pw.matches("[A-Z].*")` and so on — and confirm that the two versions agree.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a password: aubg 2026
- needs at least 10 characters (it has 9)
- needs an upper-case letter
- needs a character that is not a letter, a digit or a space
- must not contain spaces
Rejected: 4 rule(s) broken
The regular-expression checks agree: true

Enter a password: S3cure-Pass!
Accepted
The regular-expression checks agree: true
```

`matches` tests the whole string: `pw.matches("[A-Z]")` is true only for a password that is a single capital letter. The `.*` on each side is what turns "is" into "contains".

Part B · Additional Exercises

These questions go deeper into the parts of Lectures 6 and 7 that Part A only uses. Several ask you to record what the compiler or the runtime actually printed — copy those messages exactly, character for character, rather than paraphrasing them. Run them from IntelliJ, or with `javac -g` and then `java`: the source launcher compiles without local variable names, so its `NullPointerException` messages name a variable by number, such as `<local11>`, where these pages show its name (Question 23 shows both).

B1 · ARRAYS ARE REFERENCES

An array variable holds a reference, not the array. Assignment copies the reference, == compares references, and a method receives a copy of the reference — three facts that explain every question in this group.

Question 21 · One Array, Two Names

ONEARRAYTWO NAMES.JAVA

```
import java.util.Arrays;

public class OneArrayTwoNames {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = a;
        int[] c = {1, 2, 3};
        b[0] = 99;

        System.out.println(a[0]);
        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(Arrays.equals(a, c));
        c[0] = 99;
        System.out.println(Arrays.equals(a, c));
        System.out.println(a);
        System.out.println(Arrays.toString(Arrays.copyOf(a, 5)));

        char[] name = {'J', 'a', 'v', 'a'};
        System.out.println(name);
        System.out.println("Name: " + name);
    }
}
```

(a) Predict all nine lines, then run it. Two of them will not match anyone's prediction exactly: which, and why?

(b) `println(a)` prints `[I@` followed by a number. What do the `[` and the `I` stand for? Then explain why `println(name)` prints `Java` while `"Name: " + name` prints `[C@` and a number. It is an overloading question — Lab 3, Question 24.

(c) Copy a three ways — with a loop, with `System.arraycopy` and with `Arrays.copyOf` — change one element of each copy, and show that a itself is untouched. What does `Arrays.copyOf(a, 2)` give?

Question 22 · What a Method Can Change

WHAT CHANGES.JAVA

```
import java.util.Arrays;

public class WhatChanges {
    public static void main(String[] args) {
        int x = 1;
        int[] y = new int[3];
        m1(x, y);
        System.out.println("m1: x is " + x + ", y is " + Arrays.toString(y));

        int[] list = {1, 2, 3};
        m2(list);
        System.out.println("m2: " + Arrays.toString(list));
    }
}
```

```

        m3(list);
        System.out.println("m3: " + Arrays.toString(list));
        m4(list);
        System.out.println("m4: " + Arrays.toString(list));
        int[] result = m5(list);
        System.out.println("m5: " + Arrays.toString(list) + " and " + Arrays.toString(result));
    }

    public static void m1(int number, int[] numbers) {
        number = 1001;
        numbers[0] = 5555;
    }

    public static void m2(int[] arr) {
        arr = new int[] {7, 7, 7};
    }

    public static void m3(int[] arr) {
        for (int v : arr) { v = v * 2; }
    }

    public static void m4(int[] arr) {
        for (int i = 0; i < arr.length; i++) { arr[i] = arr[i] * 2; }
    }

    public static int[] m5(int[] arr) {
        int[] copy = Arrays.copyOf(arr, arr.length);
        copy[0] = -1;
        return copy;
    }
}

```

(a) Predict the five lines, then run it.

(b) `m2` assigns a new array to its parameter and the caller never sees it; `m1` assigns to an element, and the caller does. Draw the stack and the heap for both, in the style of the lecture's slide 21, and say which arrow each assignment moved.

(c) `m3` is the lecture's enhanced-for trap. In one sentence: what does `v` hold, and why does assigning to it change nothing?

(d) `m4` changes its argument; `m5` returns a changed copy and leaves its argument alone. Which would you rather call from code you did not write — and what should each one's `/** */` comment say?

B2 · INDEXES, LENGTHS AND NULL

The first week of arrays produces a short list of errors, over and over. Meet each one here on purpose, so that it is familiar when it arrives by accident.

Question 23 · Seven Mistakes, Seven Messages

Say which of the seven fragments compile. Then compile each as a class of its own, run those that compile, and copy every message.

THE SEVEN FRAGMENTS

```
// (a)
int[] list = new int[3];
list[3] = 1;

// (b)
int[] list = new int[3];
list[-1] = 1;

// (c)
int[] data = null;
data[0] = 1;
```

- (a) What two numbers does the message give you, and which one is usually the mistake?
- (b) Does a negative index wrap round to the end of the array, as in Python?
- (c) Run it twice — once after `javac -g NullArray.java`, once with the source launcher `java NullArray.java` — and compare the two messages. What does the second call the variable, and why?

```
// (d)
int[] list = {1, 2, 3};
System.out.println(list.length());

// (e)
String s = "abc";
System.out.println(s.length);

// (f)
int[] list = {3, 1, 2};
printArray(list[]);

// (g)
double[] myList;
myList = {1.9, 2.9, 3.4, 3.5};
```

(d) and (e) are the same mistake in opposite directions. Write down the rule you will use to remember which one has brackets.

- (f) Where do the brackets go when an array is passed to a method?
- (g) Three errors from one line. Which one is the real problem, and what is the one-line fix?

Question 24 · Ragged Arrays and the m[0].length Bug

RAGGED.JAVA

```
import java.util.Arrays;

public class Ragged {
    public static void main(String[] args) {
        int[][] triangle = {{1, 2, 3, 4}, {5, 6, 7}, {8, 9}, {10}};
        System.out.println(triangle.length + " " + triangle[0].length
            + " " + triangle[3].length);
        System.out.println(Arrays.deepToString(triangle));
        System.out.println("sum with m[r].length: " + sumRowByRow(triangle));

        int[][] shared = new int[2][];
        shared[0] = new int[] {1, 2};
        shared[1] = shared[0];
        shared[1][0] = 9;
        System.out.println(Arrays.deepToString(shared));

        System.out.println("sum with m[0].length: " + sumAsIfRectangular(triangle));
    }

    public static int sumRowByRow(int[][] m) {
        int total = 0;
        for (int r = 0; r < m.length; r++) {
            for (int c = 0; c < m[r].length; c++) { total += m[r][c]; }
        }
        return total;
    }

    public static int sumAsIfRectangular(int[][] m) {
        int total = 0;
        for (int r = 0; r < m.length; r++) {
            for (int c = 0; c < m[0].length; c++) { total += m[r][c]; }
        }
        return total;
    }
}
```

- (a) Predict the output up to the exception, and the exception itself. Which element was the loop trying to read when it stopped?
- (b) `shared[1] = shared[0]` turns two rows into one row with two names. Explain the printed result with a diagram of the outer array and its row references.
- (c) `new int[3][]` creates the outer array only. Print such an array with `Arrays.toString`, then assign `ragged[0][0] = 1` and copy the message.
- (d) Replace `deepToString` with `toString` in the statement that prints `triangle`, and explain what you see.

B3 · VARARGS AND COMMAND-LINE ARGUMENTS

Varargs is an array with a friendlier way of calling, and `main`'s `String[]` is an array that the launcher fills in. Both follow the rules of arrays exactly — plus a few of their own.

Question 25 · The Rules of Varargs and of `main`'s Array

TWO HEADERS

```
// header 1
public static int total(int... values, String label) {
    int sum = 0;
    for (int v : values) {
        sum += v;
    }
    return sum;
}

// header 2
public static void both(int... counts, double... prices) {
    System.out.println(counts.length + " " + prices.length);
}
```

(a) Compile both and copy the message. What is the rule, and why could the compiler not decide which arguments belong to which parameter without it?

WHICHSUM.JAVA

```
public class WhichSum {
    public static void main(String[] args) {
        System.out.println(sum(1, 2));
        System.out.println(sum(1, 2, 3));
        System.out.println(sum());
        System.out.println(sum(new int[] {4, 5}));
    }

    public static String sum(int a, int b) {
        return "sum(int, int) gives " + (a + b);
    }

    public static String sum(int... values) {
        int total = 0;
        for (int v : values) { total += v; }
        return "sum(int...) got " + values.length + " values and gives " + total;
    }
}
```

(b) Predict which overload each of the four calls runs. When a fixed-arity method and a varargs method both fit, which wins?

ECHO.JAVA

```
public class Echo {
    public static void main(String[] args) {
        System.out.println(args.length + " argument(s)");
        for (int i = 0; i < args.length; i++) {
            System.out.println("args[" + i + "] is [" + args[i] + "]");
        }
    }
}
```

(c) Run it as `java Echo.java one "two three" 4`, and then with no arguments at all. How many arguments does the first run receive, and what is `args` in the second — an empty array, or `null`?

(d) Finally, `Integer.parseInt("3.5")`. Copy the message, and name the method you would call instead.

B4 · STRINGS: EQUALITY AND IMMUTABILITY

Strings are objects, so `==` compares references, and a method cannot change the `String` it is given. Two lines of Thursday's lecture — and between them the source of most `String` bugs.

Question 26 · == Against equals

EQUALITY.JAVA

```
import java.util.Scanner;

public class Equality {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        String a = "yes";
        String b = "yes";
        String c = new String("yes");
        String d = "ye" + "s";
        String part = "ye";
        String e = part + "s";
        String f = e.intern();
        System.out.print("Type yes: ");
        String g = input.next();

        System.out.println("a == b " + (a == b));
        System.out.println("a == c " + (a == c));
        System.out.println("a == d " + (a == d));
        System.out.println("a == e " + (a == e));
        System.out.println("a == f " + (a == f));
        System.out.println("a == g " + (a == g));
        System.out.println("a.equals(c), a.equals(e) and a.equals(g) "
            + (a.equals(c) && a.equals(e) && a.equals(g)));

        String answer = null;
        System.out.println("\nyes\".equals(answer) " + "yes".equals(answer));
        System.out.println("answer.equals(\"yes\") " + answer.equals("yes"));
    }
}
```

(a) Write down all six `==` results before you run it, then run it and type `yes`. Which strings share one object in the string pool, and which were built while the program ran?

(b) Make part final — `final String part = "ye";` — and run it again. Which line changes, and why can the compiler now work out `part + "s"` by itself?

(c) The last line throws. Copy the message, and explain why the line above it, with the literal first, cannot throw.

Question 27 · Changing a String That Cannot Change

IMMUTABLE.JAVA

```
public class Immutable {
    public static void main(String[] args) {
        String s = "java";
        s.toUpperCase();
        s.concat(" rocks");
        s.replace('j', 'J');
        System.out.println(s);
        s = s.toUpperCase();
        System.out.println(s);

        StringBuilder sb1 = new StringBuilder("abc");
        StringBuilder sb2 = new StringBuilder("abc");
        System.out.println(sb1.equals(sb2));
        System.out.println(sb1.toString().equals(sb2.toString()));
        System.out.println(sb1.compareTo(sb2) == 0);

        addStar(s);
        addStar(sb1);
        System.out.println(s + " " + sb1);
    }

    public static void addStar(String text) {
        text = text + "*";
    }

    public static void addStar(StringBuilder text) {
        text.append('*');
    }
}
```

(a) Predict the six lines, then run it. Which three statements in the first block do nothing at all — and does the compiler warn you?

(b) `sb1.equals(sb2)` is false for two builders holding the same text. What does `StringBuilder`'s `equals` compare, and which two lines show the correct ways to compare contents?

(c) The two `addStar` methods are overloads (Lab 3, Question 24). Explain why one changes what the caller sees and the other does not, when both look as if they add a star.

B5 · COMPARING, INDEXING AND SPLITTING

`compareTo`, `index` arithmetic and `split` each have one rule that surprises people exactly once. These two questions arrange for that once to be now.

Question 28 · `compareTo` and the Index Ruler

COMPAREANDINDEX.JAVA

```
import java.util.Arrays;

public class CompareAndIndex {
    public static void main(String[] args) {
        System.out.println("Apple".compareTo("apple"));
        System.out.println("apple".compareTo("Apple"));
        System.out.println("abc".compareTo("abcd"));
        System.out.println("abc".compareTo("abx"));
        System.out.println("Zebra".compareTo("apple") < 0);
        System.out.println("Zebra".compareToIgnoreCase("apple") < 0);

        String[] names = {"zoe", "Adam", "bob", "Eve"};
        Arrays.sort(names);
        System.out.println(Arrays.toString(names));

        String s = "Programming";
        System.out.println(s.indexOf('g') + " " + s.lastIndexOf('g') + " " + s.indexOf('z'));
        System.out.println(s.substring(3, 7));
    }
}
```

(a) Predict the nine lines. Why is `"Apple".compareTo("apple")` -32, and why does "Zebra" sort before "apple"?

(b) A student writes `if (a.compareTo(b) == -1)` to mean "a comes first". Find a pair of strings in this program for which that test gives the wrong answer.

Each fragment below stops with an exception. Run it and copy the first line of the message.

THREE FRAGMENTS

```
// (c)
String s = "Programming";
System.out.println(s.charAt(s.length()));

// (d)
String s = "Programming";
System.out.println(s.substring(3, 12));

// (e)
String s = "Programming";
System.out.println(s.substring(s.indexOf('z')));
```

(c) Which index does `charAt` accept last, and how could you tell from the message?

(d) How does `substring`'s message show that its end index is exclusive?

(e) What went wrong before `substring` was even called?

Question 29 · Three split Traps and a Dot

SPLITTRAPS.JAVA

```
import java.util.Arrays;

public class SplitTraps {
    public static void main(String[] args) {
        show("1.2.3".split("."));
        show("1.2.3".split("\\."));
        show(",a,b".split(","));
        show("a,b,.".split(","));
        show("a,b,.".split(",", -1));
        show("a|b|c".split("|"));
        show("a b c".split(" "));
        show("a b c".split("\\s+"));
    }

    public static void show(String[] parts) {
        System.out.println(parts.length + " " + Arrays.toString(parts));
    }
}
```

(a) Predict the length and contents of all eight results, then run it. Explain the first and the sixth — both are about what the argument of `split` really is.

(b) The third and fourth lines treat empty strings differently at the two ends. State the rule, and say what the -1 in the fifth line changes.

DATEPATTERN.JAVA

```
public class DatePattern {
    public static void main(String[] args) {
        String escaped = "\\d{2}\\d{2}\\d{4}";
        String unescaped = "\\d{2}\\.\\d{2}\\d{4}";
        System.out.println("29.09.2026".matches(escaped));
        System.out.println("29-09-2026".matches(escaped));
        System.out.println("29.09.2026".matches(unescaped));
        System.out.println("29-09-2026".matches(unescaped));
        System.out.println("29.09.26".matches(escaped));
        System.out.println("COS 2030, INF 1030".replaceAll("\\d", "#"));
    }
}
```

(c) Predict the six lines. Which pattern accepts a date written with hyphens, and why? What does the last line show about the first argument of `replaceAll`?

B6 · BUILDING STRINGS

A String cannot change, so `s += "x"` builds a new one every time round the loop. How much that costs is not a matter of opinion; it can be counted.

Question 30 · The Cost of += in a Loop

BUILDCOST.JAVA

```
public class BuildCost {
    public static void main(String[] args) {
        final int N = 10_000;

        String s = "";
        long built = 0;
        for (int i = 0; i < N; i++) {
            s += "x";
            built += s.length();
        }
        System.out.println("+= built " + N + " strings, " + built + " characters in all");

        StringBuilder sb = new StringBuilder();
        int capacity = sb.capacity();
        int growths = 0;
        System.out.print("StringBuilder capacity: " + capacity);
        for (int i = 0; i < N; i++) {
            sb.append('x');
            if (sb.capacity() != capacity) {
                capacity = sb.capacity();
                growths++;
                System.out.print(" -> " + capacity);
            }
        }
        System.out.println();
        System.out.println("It grew " + growths + " times; same text: "
            + s.equals(sb.toString()));
    }
}
```

EXPECTED OUTPUT

```
+= built 10000 strings, 50005000 characters in all
StringBuilder capacity: 16 -> 34 -> 70 -> 142 -> 286 -> 574 -> 1150 -> 2302 -> 4606 -> 9214 -> 18430
It grew 10 times; same text: true
```

- (a) Explain the 50,005,000: how many characters did the += loop copy in all, and what formula gives that number for $n = 10,000$?
- (b) The StringBuilder grew ten times. What rule takes each capacity to the next, and roughly how many characters did all ten growths copy?
- (c) Time both loops with `System.nanoTime()` for $n = 100,000$ and run the program three times. Why is the ratio different every time, and why is the character count the better argument?